

Verteilte Systeme Übungsblatt 2

Aufgabe 1

Wahl der Programmiersprache

Die Wahl der Programmiersprache ist für dieses Übungsblatt auf Kotlin gefallen. Als moderne JVM-Sprache ist sie vollständig mit dem bestehenden Framework kompatibel. Kotlin-Klassen kompilieren genauso wie Java-Klassen zu JVM-Bytecode, Java-Bibliotheken können direkt in Kotlin verwendet werden. Es entsteht so 100 % Interoperabilität, ohne zusätzlichen Installationsaufwand, wie er bei Superscript-Sprachen auftreten würde.

Kotlin verfügt über moderne Sprachfeatures, wie man sie aus anderen Java-überlegenen Sprachen wie JavaScript kennt, zum Beispiel Typinferenz, Smart Casts, Extension Functions. Das macht die Entwicklung mit Kotlin deutlich angenehmer. Weitere Features wie der Elvis-Operator `?:` und verwandte Null-Safety-Operatoren verhindern zudem leidige Null-Pointer-Exceptions und reduzieren aufwändiges Error-Handling auf wenige Zeichen. Insgesamt benötigen Kotlin-Klassen bedeutend weniger Boilerplate-Code, was zusammen mit der modernen Syntax der Sprache für verbesserte Lesbarkeit sorgt.

Struktur der Implementierung

Um die drei Distributed-Shared-Memory-Varianten effizient zu implementieren, wird auf einem Interface *DSMNode* aufgebaut. Dieses definiert die beiden Grundfunktionen jeder Node: *Read()*, *Write()* und die zum Monitoring benötigten Funktionen *NodeName()* und *getLocalState()*.

Am einfachsten ist die Implementierung der AP-Node. Diese soll immer verfügbar sein (Availability) und bei Partition weiter funktionieren (Partition Tolerance). Jede Node hält dazu eine lokale Kopie der Key-Value-Paare aller zum System gehörenden Nodes. Für AP wird nur eine einzige Art an Nachricht verwendet. *WriteMessage* definiert eine Erweiterung der Klasse *Message* des Frameworks so, dass jeweils ein Schlüssel, ein Wert und ein Zeitstempel versandt werden. Es reicht hier ein einfacher Zeitstempel der Systemzeit, da das Verteilte System lokal simuliert wird. Die Systemzeit ist damit auf allen Nodes exakt gleich, eine separate logische Uhr muss also nicht implementiert werden.

Die Klasse implementiert das *DSMNode*-Interface und damit auch mindestens die vier definierten Funktionen.

write(key, value) schreibt dabei einfach den der Funktionen übergebenen Werte in den lokalen Speicher und teilt diese Aktualisierung den anderen Nodes per Broadcast einer Nachricht vom vorher definierten Typ *WriteMessage* mit. Es wird keine Bestätigung abgewartet.

Ebenso gibt die Funktion *read(key)* schlicht den entsprechenden Wert aus dem lokalen Speicher aus.

Die beiden helper-Funktionen geben schlicht den geforderten Wert zurück. Im Fall von *nodeName()* ist das der Name der Node, bei *getLocalState(key)* der mittels *Read*-Aufruf ermittelte Inhalt der entsprechenden Zelle.

engage() ist die Funktion, die beim Start des Simulators automatisch ausgeführt wird. Sie behandelt eingehende Nachrichten anderer Nodes und übernimmt deren geschriebenen Werte in den lokalen Speicher, sofern der vorhanden Wert älter ist als der neu empfangene. Das sorgt dafür, dass im Falle eines Verzögerten Eintreffens von Nachrichten keine Inkonsistenzen durch Sprünge zurück entstehen.

Etwas komplizierter ist die CA-Node (Konsistenz und Verfügbarkeit). Sie erweitert die bestehende Implementierung um den Nachrichtentyp *WriteAck* und baut *WriteMessage* zu *WriteRequest* um, indem zusätzlich eine ID zur Identifizierung des Request sowie der Namen des Absenders hinzugefügt werden. Diese dienen im Mainloop *engage()* dazu, zuordbare Bestätigungen zu versenden.

Die Schreibfunktion wird um ein Quorum erweitert. Jeder Schreibvorgang muss von einer errechneten Anzahl an Nodes quittiert werden, bevor er als ausgeführt gilt, bis das passiert ist blockiert der Thread und lässt keine anderen Aktionen zu. Die Größe dieses Quorums entspricht dabei der Hälfte der Anzahl der Nodes plus der Absender der Anfrage selbst.

Die restlichen Funktionen bleiben bei CA unberührt, werden aber für CP angepasst.

Für Konsistenz und Partitionstoleranz übernehmen wir das quorumsbasierte Schreiben aus CA und bauen analog ein quorumsbasiertes Lesen ein.

An dieser Stelle entsteht leider ein Problem, das es mir nicht gelang zu lösen. Sobald auf einem CP-Netzwerk getestet wird, entsteht durch die zeitgleiche Verwendung von Quorumslesen und -schreiben ein Deadlock, der dazu führt, dass der Test nicht mehr terminiert. Beispielsweise kann Node 1 einen *WriteRequest* senden und dann auf eine Bestätigung von Node 2 oder 3 warten, währenddessen hat der Beobachter des Tests eine Leseanfrage auf Nodes 2 und 3 initialisiert und wartet dort auf Bestätigung von 1. Das Resultat: Das Programm bleibt stecken, alle Nodes blockieren, weil sie auf eine Antwort warten.

Um das zu umgehen, ruft die Funktion *getLocalState()* diesmal nicht die *Read*-Methode auf, sondern gibt schlicht den lokalen Stand der jeweiligen Node zurück.

Dieser zuerst clever wirkenden Workaround entpuppt sich später als fatal. Dazu mehr in der Auswertung der Ergebnisse.

Aufgabe 2 und 3

Als Beispiel-Anwendung habe ich einen simplen Counter implementiert. Jede Node zählt in denselben Abständen hoch und teilt dem Rest des Netzwerks die eigenen Änderungen mit, empfängt aber zugleich auch deren Änderungen. Es wird also immer parallel geschrieben und gelesen, das anspruchsvollste Zugriffsmuster.

Zeitgleich werden vier Tests definiert um Daten für die Auswertung in Aufgabe drei zu sammeln. Der erste Test geht von einer niedrigen Latenz ohne Partitionierung aus. Die Latenz wird durch die standardmäßig verwendete FiFo-Schlange simuliert. Die Nachrichten kommen also in der Reihenfolge an, in der sie versendet werden.

Um im zweiten Test eine statische Partitionierung zu simulieren, implementiere ich eine eigene Liste, die die bestehenden Simulation des Netzwerk erweitert, sodass Nachrichten, die aus der anderen Partition stammen, herausgefiltert werden können. Diese Partitionierung kann simpel während der Simulation aufgehoben werden. Im Test passiert das nach der Hälfte der Laufzeit.

Zu beiden Test, sowohl mit als auch ohne Partitionierung, gibt es eine zweite Version mit hoher Latenz. Diese wird dadurch simuliert, dass eine uniforme Verteilung der Nachrichten in den Warteschlangen angenommen wird. Alte Nachrichten werden also mit derselben Wahrscheinlichkeit zugestellt wie neue Nachrichten. Die Reihenfolge wird dabei nicht gewahrt.

Simuliert wird jeder Test für jeden Typ jeweils 30 Sekunden, wobei sekundlich ein Snapshot erstellt wird.

Dieser erfragt von jeder Node den Wert zu jeder Node und erstellt eine Matrix als Ausgabe, wobei verglichen wird, ob die gespeicherten Daten an allen Nodes gleich, also konsistent, sind.

Ergebnisse

Die Ergebnisse fallen leider durchwachsen aus.

Für die Test mit Partitionierung wird Konsistenz nur innerhalb einer Partition angenommen.

Die AP-Implementierung erreicht sowohl bei geringer als auch hoher Latenz ohne Partition eine Konsistenz von ca. 90%. Für die Test mit Partition sind die Werte identisch oder besser. Dieser Effekt entsteht dadurch, dass die Konsistenz in der ersten Hälfte des Tests nur auf einer Teilmenge erwartet wird, durch die entsprechend geringer Gesamtmenge an beteiligten Nodes sinkt so die Wahrscheinlichkeit in einen Fehler zu laufen.

Sobald die Partitionen zusammengelegt werden, empfangen alle Nodes wie erwartet die aktuellen Werte und sind so schnell konsistent, dass in der Zwischenzeit kein Snapshot gemacht werden kann.

Obwohl die Konsistenz nicht perfekt ist, ist sie für AP-Erwartungen hoch. Dies ist zweifelsohne auf die beinahe sofortige Zustellung der Nachrichten zurückzuführen. Die simulierte hohe Latenz ändert daran wenig. Obwohl manche Nachrichten nun länger brauchen als andere, ist die tatsächliche Auswirkung auf das Experiment aufgrund des geringen Nachrichtenaufkommens kaum bemerkbar. Der erzeugte Jitter wirkt sich aufgrund der Nutzung des zeitlich jüngsten statt des zuletzt eingetroffenen Werts ebenfalls nicht auf das Resultat aus.

Bei Betrachtung der CP- und CA-Node fällt auf, dass die Ergebnisse stark von den Erwartungen abweichen. Beide erreichen ohne Partition häufig Werte unter 85% Konsistenz, während 100% Konsistenz erwartet waren. Das ist darauf zurückzuführen, dass aus dem lokalen Speicher gelesen wird. Inkonsistenzen treten selbst bei geringer Latenz ohne Partition auf. Das ist ein Indiz dafür, dass die Operationen Timeouts und nicht atomar ausgeführt Operationen reichen, um die hohen Konsistenzversprechen zu verhindern. Die Timeouts sind schließlich eine explizite Entscheidung für Verfügbarkeit auf Kosten der Konsistenz, um eine sinnvolle Durchführung der Tests zu erlauben.

Wenn diese Systeme wirklich stark konsistent sein müssten, dürften sie bei fehlendem Quorum/Ack niemals ein Timeout haben, sondern müssten ewig blockieren. Das führt in der Praxis leider dazu, dass die Tests ohne Ausgabe bleiben und nicht terminieren.

Dieser Bruch mit den Erwartungen setzt sich auch bei den Tests mit Partition fort. Bei beiden ist zu beobachten, dass anfänglich blockiert wird. Die Zähler steigen nicht, weil die Partitionsgröße unter der Quorums-Größe liegt, und so nie ein Schreibvorgang genügend quittiert wird. Durch die gezwungenermaßen implementierten Timeouts zählen die Nodes aber schließlich doch hoch, nur eben langsamer.

Nach Zusammenlegung der Partitionen verlaufen die Tests wie zuvor beobachtet.

Zu erwarten war eigentlich, dass lediglich bei AP Inkonsistenzen auftreten, CP und CA dafür häufiger blockieren. Dies trifft nicht ein. Die Diskrepanz ist auf die suboptimale Implementierung der beiden letzten Varianten zurückzuführen. Die Ergebnisse für AP befinden sich im Rahmen der Erwartungen.

In der Theorie hätten die Experimente zeigen sollen, dass AP sich besonders dort eignet, wo Verfügbarkeit wichtig ist, Inkonsistenzen aber verkraftet werden können. Als Beispiel bieten sich Social-Media-Feeds an. Der Nutzer will immer Zugriff haben, Daten wie Anzahl der Likes oder Kommentare müssen dafür nicht immer topaktuell sein.

CP bietet sich dafür bei Systemen an, deren Korrektheit kritisch ist. Beispielsweise Banksysteme, hier sollten keine Doppelbuchungen stattfinden können.

CA bietet dabei das Beste aus beiden Welten, sofern davon auszugehen ist, dass keine Partitionierung auftreten kann. Es könnte für zum Beispiel für lokale Datenbanken verwendet werden, eignet sich aber aufgrund besagter Einschränkung weniger für tatsächlich verteilte Systeme.